

File 1 — retrieval/schema_retriever.py

BEFORE:

```
BANK_TERMS = {
    "bank", "account", "ifsc", "dbt", "payment",
    "sbi", "pnb", "bob", "hdfc", "icici", "uco", "canara", "baroda",
    "gramin", "cooperative", "union bank", "central bank", "indian bank",
}
```

AFTER:

```
BANK_TERMS = {
    "bank", "account", "ifsc", "dbt", "payment",
    "sbi", "pnb", "bob", "hdfc", "icici", "uco", "canara", "baroda",
    "gramin", "cooperative", "union bank", "central bank", "indian bank",
    "unbanked", "no bank", "without bank", "no account", "without account",
}
```

File 2 — prompting/prompt_builder.py

BEFORE:

```
if "bank_details.bank_name" in result.columns:

    dynamic_rules.append(

        "- bank_name: Always use UPPER(bank_name) LIKE '%SEARCH_TERM_UPPER%'.\n"

        " * If filtering by district too, JOIN family table."

    )
```

AFTER:

```
if "bank_details.bank_name" in result.columns:
    dynamic_rules.append(
        "- bank_name: Always use UPPER(bank_name) LIKE '%SEARCH_TERM_UPPER%'.\n"
        " * If filtering by district too, JOIN family table."
    )

# Add this new block right after
if "bank_details.bank_id" in result.columns or "bank_details.bank_name" in result.columns:
    q_lower = result.question.lower()
    if any(word in q_lower for word in ["no bank", "without bank", "don't have",
                                         "do not have", "no account", "unbanked",
                                         "without account"]):
        dynamic_rules.append(
            "- no bank account query: Use LEFT JOIN bank_details ON "
            "bank_details.member_id = member.member_id "
            "WHERE bank_details.bank_id IS NULL. "
            "Never use INNER JOIN for this — INNER JOIN returns zero rows."
        )
```

File 3 — app.py

BEFORE — module level (nothing):

```
# Pre-build district lookups once at module level
_DISTRICTS_LOWER = {d.lower() for d in RAJASTHAN_DISTRICTS_41}
_DISTRICT_CANONICAL = {d.lower(): d for d in RAJASTHAN_DISTRICTS_41}
```

AFTER — two new constants added:

```
# Pre-build district lookups once at module level
_DISTRICTS_LOWER = {d.lower() for d in RAJASTHAN_DISTRICTS_41}
_DISTRICT_CANONICAL = {d.lower(): d for d in RAJASTHAN_DISTRICTS_41}

# Words that indicate a "no bank account" query
_NO_BANK_WORDS = [
    "no bank", "without bank", "don't have", "do not have",
    "no account", "unbanked", "without account",
]

# Words that signal the user explicitly wants a specific member_type filter
_MEMBER_TYPE_WORDS = [
    "regular member", "mem type", "member type", "hof", "head of family",
]
```

BEFORE — `_post_process_sql` had 15 steps, no no-bank logic. `generate_sql_pipeline` was:

```
for _ in range(settings.max_retries):
    prompt = prompt_builder.build(retrieval, previous_error=previous_error)
    sql = generator.generate(prompt)
    sql = _post_process_sql(sql)
    validation = validator.validate(...)
```

AFTER — new `_fix_no_bank_sql` function added between `_post_process_sql` and `generate_sql_pipeline`:

```
def _fix_no_bank_sql(sql: str, question: str) -> str:
    if not any(w in question.lower() for w in _NO_BANK_WORDS):
        return sql

    # If LLM forgot bank_details JOIN entirely, inject it
    if "bank_details" not in sql.lower():
        sql = re.sub(
            r"\b(FROM\s+member(?:\s+(?:AS\s+)?\w+)?)\b",
            r"\1 LEFT JOIN bank_details ON bank_details.member_id = "
            + member.member_id + ",\n"
            + sql, flags=re.IGNORECASE, count=1,
        )

    # Fix INNER JOIN bank_details → LEFT JOIN
    sql = re.sub(
        r"\bINNER\s+JOIN\s+(bank_details\b)",
        r"LEFT JOIN \1",
        sql, flags=re.IGNORECASE,
    )

    # Fix plain JOIN bank_details → LEFT JOIN
    sql = re.sub(
        r"(?!LEFT\s)(?!\bLEFT)\bJOIN\s+(bank_details\b)",
        r"LEFT JOIN \1",
```

```

        sql, flags=re.IGNORECASE,
    )

    # Add IS NULL check if not already present
    if "bank_details" in sql.lower() and "IS NULL" not in sql.upper():
        if re.search(r"\bWHERE\b", sql, re.IGNORECASE):
            sql = re.sub(
                r"\bWHERE\b",
                "WHERE bank_details.bank_id IS NULL AND",
                sql, flags=re.IGNORECASE, count=1,
            )
        else:
            sql = re.sub(
                r"\b(GROUP\s+BY|ORDER\s+BY|LIMIT)\b",
                r"WHERE bank_details.bank_id IS NULL \1",
                sql, flags=re.IGNORECASE, count=1,
            )
            if "bank_details.bank_id IS NULL" not in sql:
                sql = sql.rstrip(";") + " WHERE bank_details.bank_id IS NULL;"

    # Remove spurious member_type = 'MEM' ONLY when user did not ask for it
    if not any(w in question.lower() for w in _MEMBER_TYPE_WORDS):
        sql = re.sub(
            r"\s+AND\s+(?:\w+\.)?member_type\s*=\s*'MEM'",
            "",
            sql, flags=re.IGNORECASE,
        )

    return sql

```

BEFORE — generate_sql_pipeline loop:

```

for _ in range(settings.max_retries):
    prompt = prompt_builder.build(retrieval, previous_error=previous_error)
    sql = generator.generate(prompt)
    sql = _post_process_sql(sql)
    validation = validator.validate(
        sql,
        allowed_tables=retrieval.tables,
        allowed_columns=retrieval.columns,
    )

```

AFTER — one new line added:

```

for _ in range(settings.max_retries):
    prompt = prompt_builder.build(retrieval, previous_error=previous_error)
    sql = generator.generate(prompt)
    sql = _post_process_sql(sql)
    sql = _fix_no_bank_sql(sql, question)    # ← new line
    validation = validator.validate(
        sql,
        allowed_tables=retrieval.tables,
        allowed_columns=retrieval.columns,
    )

```